



ML & DL

Lab - 10

Vishal Baraiya

Step 1: Import Libraries

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_moons, make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
```

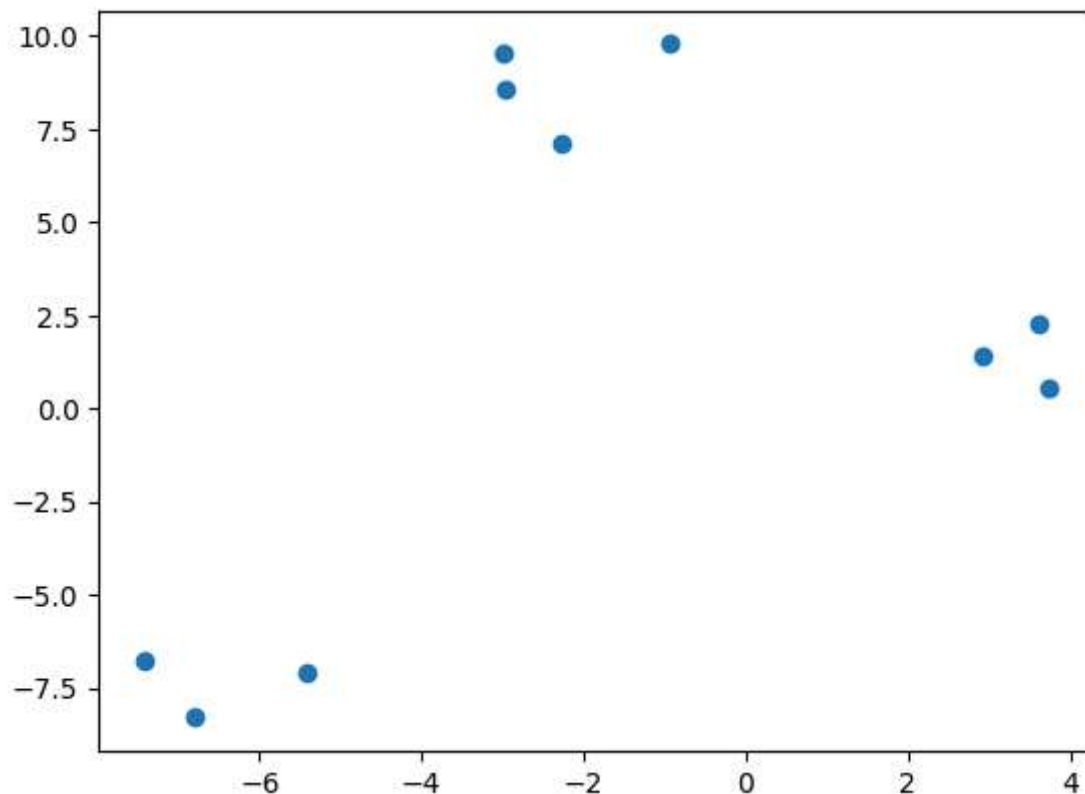
Step 2: Generate dummy data using make_blobs

```
In [2]: x,y = make_blobs(n_samples=10, random_state=42,cluster_std=1,)
```

```
In [ ]:
```

Step 3: Plot Generated data

```
In [3]: plt.scatter(x[:, 0], x[:, 1])  
plt.show()
```



Step 4: Apply Hierarchical Clustering

```
In [4]: from sklearn.cluster import AgglomerativeClustering
```

```
In [5]: model = AgglomerativeClustering(n_clusters=4,)
```

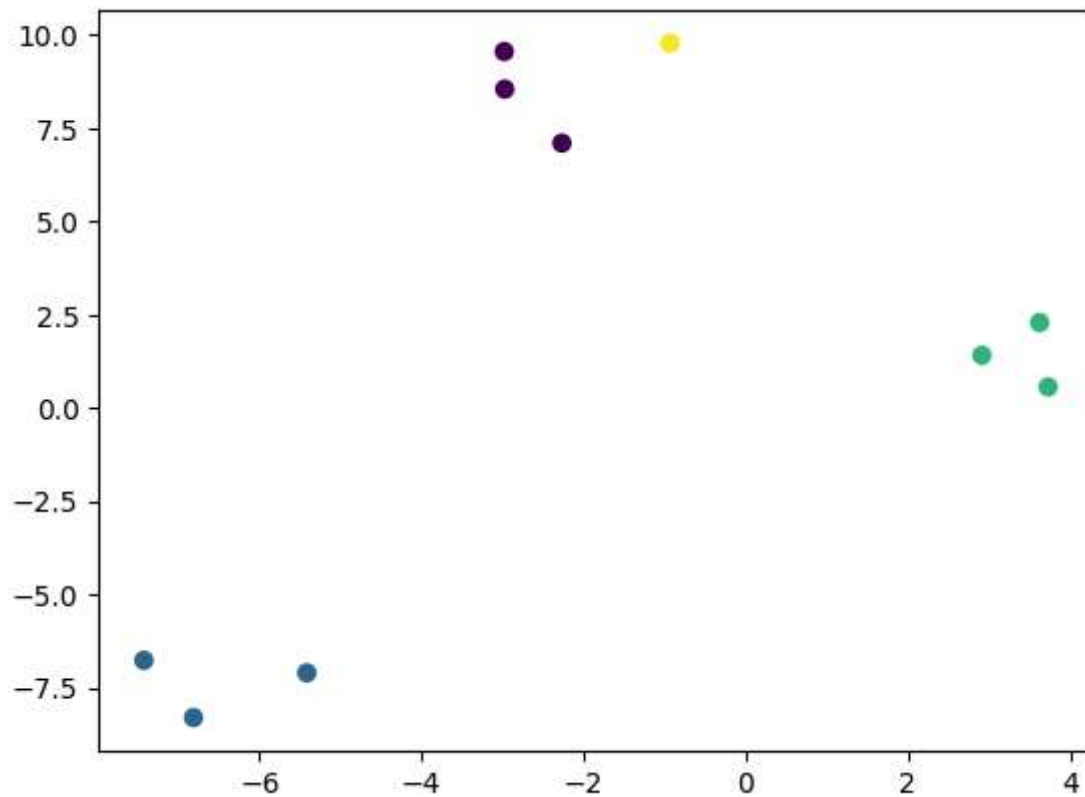
```
In [6]: model.fit(x)
```

```
Out[6]: ▼ AgglomerativeClustering ⓘ ?  
        ► Parameters
```

```
In [7]: model.labels_
```

```
Out[7]: array([1, 1, 2, 1, 0, 0, 3, 2, 2, 0])
```

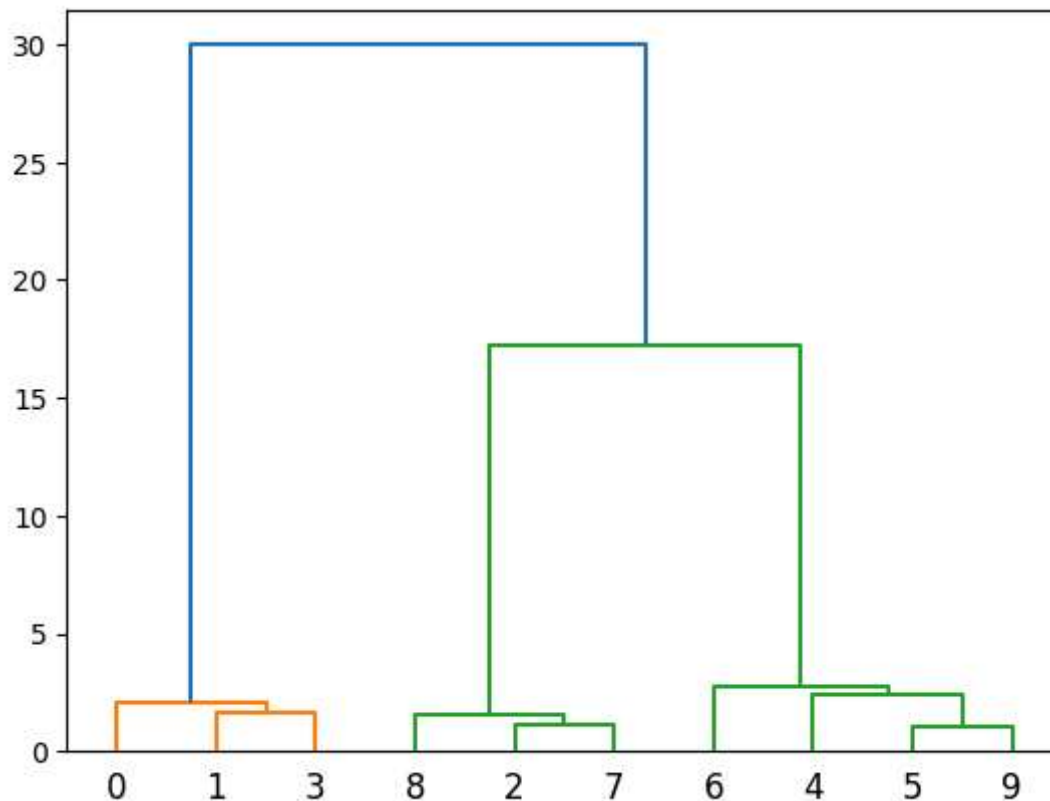
```
In [8]: plt.scatter(x[:, 0], x[:, 1], c=model.labels_)  
plt.show()
```



Step 5: Display Dendrogram

```
In [9]: from scipy.cluster.hierarchy import dendrogram, linkage
```

```
In [10]: linked = linkage(x,method='ward')  
dendrogram(linked)  
plt.show()
```

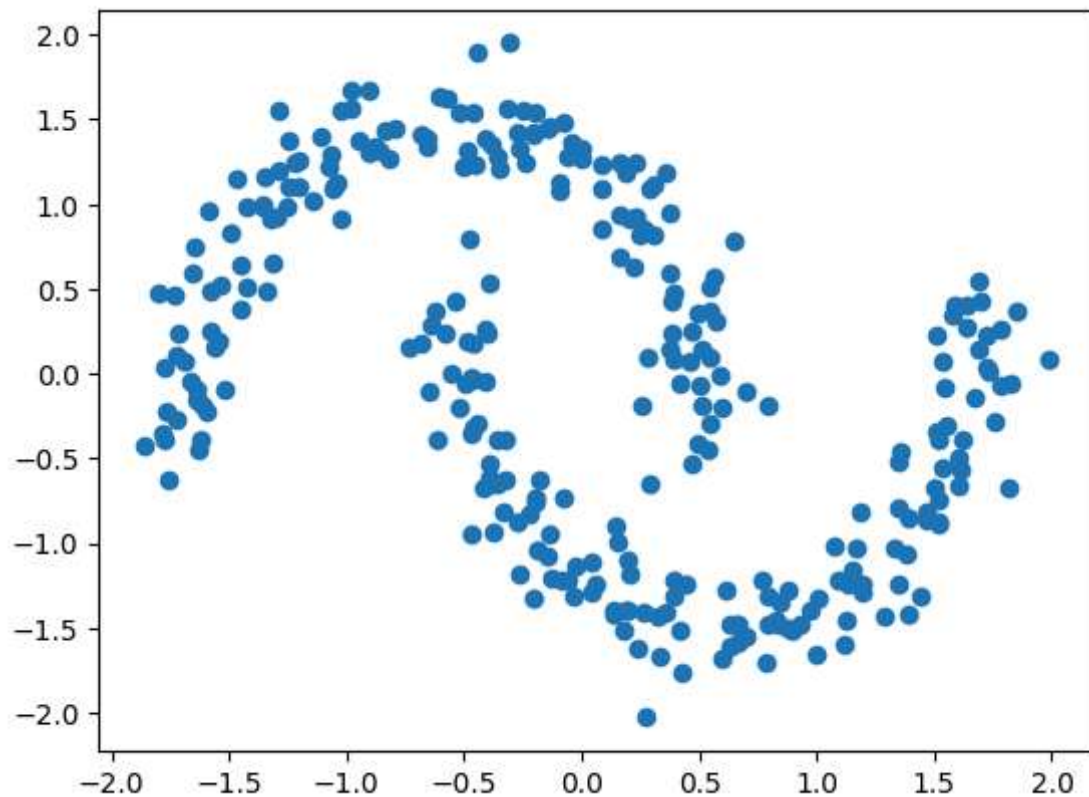


Step 6: Generate dummy data using make_moons

```
In [11]: x, y = make_moons(n_samples=300, noise=0.09, random_state=50)
x = StandardScaler().fit_transform(x)
```

Step 7: Plot Generated data

```
In [12]: plt.scatter(x[:, 0], x[:, 1],)
plt.show()
```



Step 8: apply DBSCAN Clustering

```
In [13]: from sklearn.cluster import DBSCAN
```

```
In [14]: dbscan = DBSCAN(eps=0.2, min_samples=5)  
dbscan.fit(x)
```

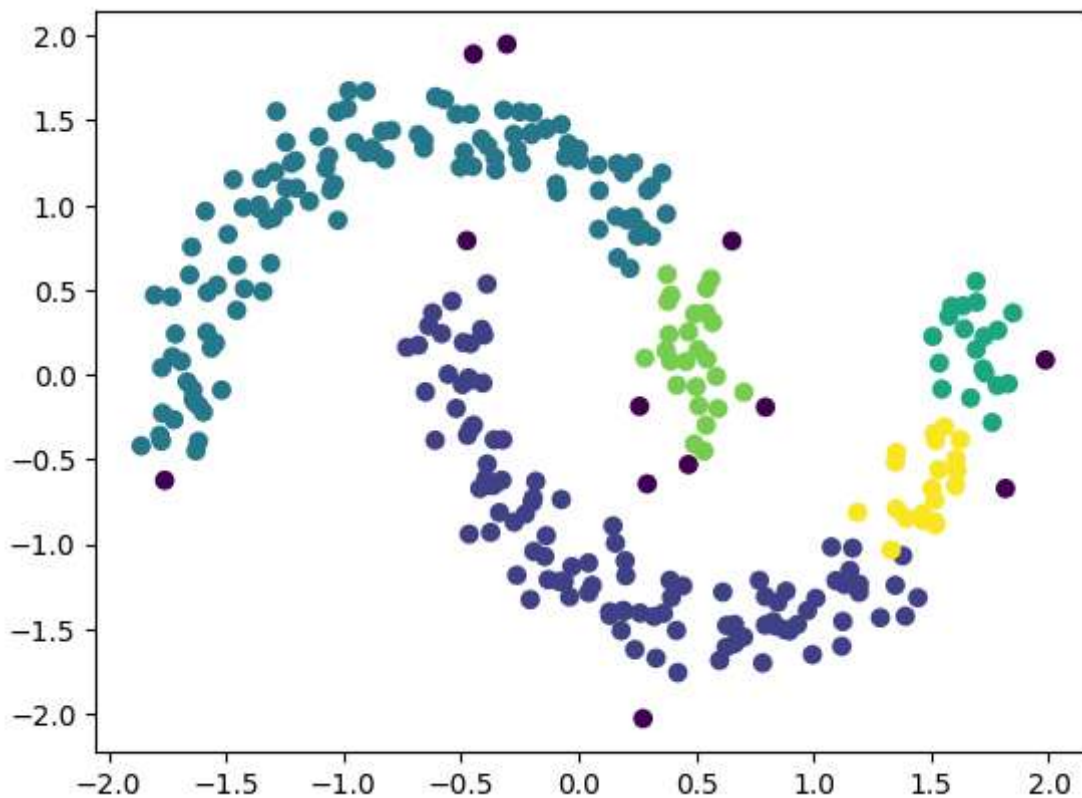
```
Out[14]: ▾ DBSCAN ⓘ ?  
          ► Parameters
```

```
In [15]: dbscan.labels_
```

```
Out[15]: array([ 0,  1,  0,  0,  1,  1,  1,  0,  1,  1,  1,  2,  2,  1,  0,  0,  1,
                0,  0,  1,  1,  1,  3,  3,  0, -1,  1,  4,  0,  1,  1,  1,  1,  3,
                1,  1,  0,  0,  2,  0,  0,  3,  0,  3, -1,  0,  1,  0,  0,  1,  1,
                3,  3,  2, -1,  1,  1,  0,  0,  0,  2,  1,  1,  0,  0,  1,  1,  0,
                1,  0, -1,  1,  1,  3,  0,  0, -1,  0,  1,  1,  0, -1,  4,  1,  0,
                3,  1,  1,  1,  0,  0,  4,  2,  1,  0,  0,  4,  0,  4,  1,  1,  0,
                0,  1,  1,  1,  1,  0,  0,  2,  0,  4,  0,  3,  0,  0,  0, -1,  0,
                1,  0,  1,  1,  0,  4,  2,  0,  1,  0,  1,  1,  0,  3,  0,  0,  1,
                0,  1,  2,  0,  0,  4,  4,  0,  4,  0,  3,  1,  1,  3,  0,  1,  1,
                0,  0,  1,  1, -1,  0,  0,  4,  1,  0,  1,  0, -1,  1,  3,  0,  1,
                0,  0,  1,  0,  4,  3,  2,  4,  0, -1,  0,  3,  1,  2,  0,  0,  3,
                1,  0,  0,  3,  4,  3,  1,  1,  1,  2,  1,  0,  1,  1,  0,  0,  3,
                1,  0,  0,  0,  1,  3,  1,  0,  0,  0,  1,  1,  0,  0,  0,  1,  3,
                1,  1,  1,  0,  1,  0,  1,  2,  1,  0,  1,  3,  1,  1,  1,  0,  1,
                3,  3,  0,  1,  0,  1,  1,  4,  2,  0,  1,  0,  1,  0,  1,  0,  1,
                1,  0,  2,  0,  0,  1,  2,  2,  1,  4,  1,  0,  1, -1,  1,  4,  4,
                1,  1,  1,  1,  1,  1, -1,  1,  1,  1,  0,  0,  1,  0,  1,  0,  1,
                4,  1,  0,  4,  2,  0,  1,  1,  2,  1,  0])
```

```
In [16]: plt.scatter(x[:, 0], x[:, 1], c=dbscan.labels_,)
```

```
Out[16]: <matplotlib.collections.PathCollection at 0x13a823d9f90>
```



Step 9: apply K Mean Clustering on same data

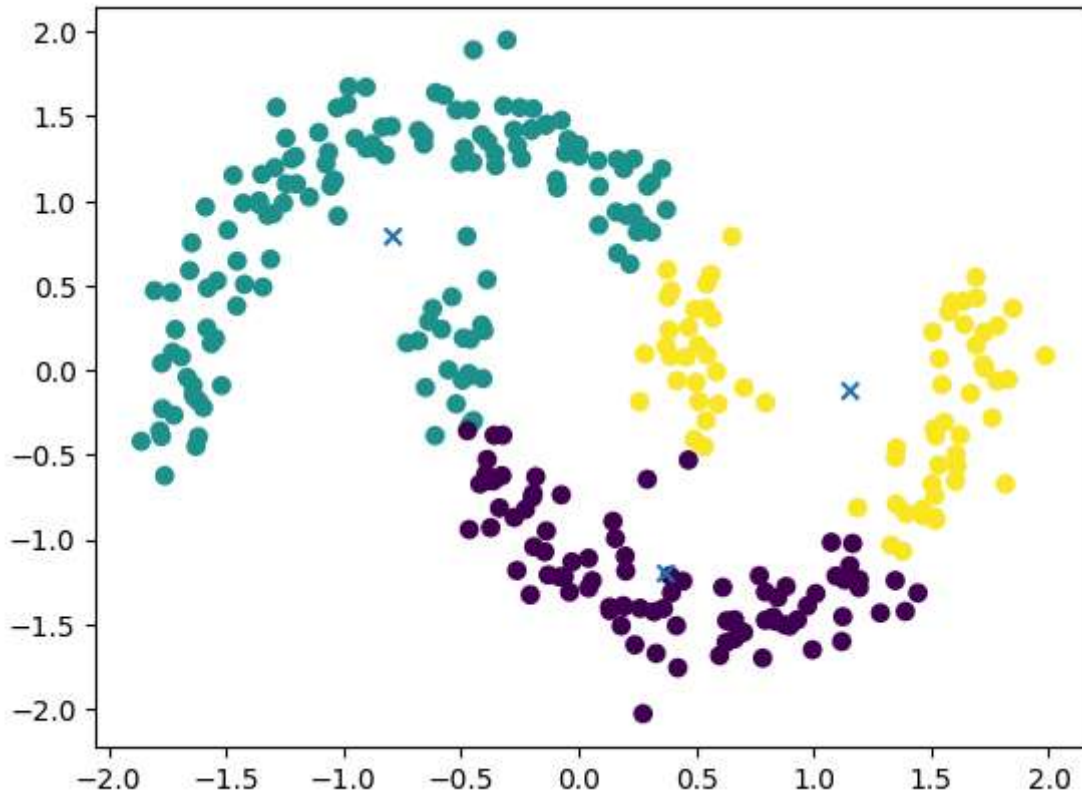
```
In [17]: from sklearn.cluster import KMeans
```

```

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(x)
centerdata = kmeans.cluster_centers_

plt.scatter(x[:, 0], x[:, 1], c=kmeans_labels)
plt.scatter(centerdata[:, 0], centerdata[:, 1], marker="x")
plt.show()

```



```
In [18]: kmeans.cluster_centers_
```

```

Out[18]: array([[ 0.36753391, -1.19748846],
                [-0.79461169,  0.79992829],
                [ 1.14988393, -0.1172976 ]])

```

Mall Customers Segmentation using DBSCAN

Perform Following steps on Dataset for customer segmentation

- > Load & Explore the Dataset
- > Data Preprocessing
- > Apply DBSCAN
- > Visualizing Customer Segments
- > Outliers (high-spending VIP customers or unusual shoppers)

```
In [19]: df = pd.read_csv("Mall_Customers.csv")
```

```
In [20]: df
```

```
Out[20]:
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

200 rows × 5 columns

```
In [21]: features = ['Annual Income (k$)', 'Spending Score (1-100)']
x = df[features]
```

```
In [22]: df['gender_'] = df['Gender'].map({'Female':0, 'Male':1})
```

```
In [23]: df
```

Out[23]:

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	gender_
0	1	Male	19	15	39	1
1	2	Male	21	15	81	1
2	3	Female	20	16	6	0
3	4	Female	23	16	77	0
4	5	Female	31	17	40	0
...	...	...	...	...	...	...
195	196	Female	35	120	79	0
196	197	Female	45	126	28	0
197	198	Male	32	126	74	1
198	199	Male	32	137	18	1
199	200	Male	30	137	83	1

200 rows × 6 columns

In [24]: df

Out[24]:

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	gender_
0	1	Male	19	15	39	1
1	2	Male	21	15	81	1
2	3	Female	20	16	6	0
3	4	Female	23	16	77	0
4	5	Female	31	17	40	0
...	...	...	...	...	...	...
195	196	Female	35	120	79	0
196	197	Female	45	126	28	0
197	198	Male	32	126	74	1
198	199	Male	32	137	18	1
199	200	Male	30	137	83	1

200 rows × 6 columns

```
In [1]: scaler = StandardScaler()
df_scaled = scaler.fit_transform(df_selected)
```

```
In [ ]: dbscan = DBSCAN(eps=0.5, min_samples=5)
df['cluster'] = dbscan.fit_predict(df_scaled)
```

```
In [ ]: plt.figure(figsize=(10, 6))
sns.scatterplot(x=df['Annual Income (k$)'], y=df['Spending Score (1-100)'], hue=df['cluster'])
plt.title("DBSCAN ")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.show()
```

```
In [ ]: df[df['cluster'] == -1]
```

```
In [ ]:
```

```
In [ ]: gdf = df.groupby('cluster')
```

```
In [ ]: for no,data in gdf:
    print(no)
    print(data['Annual Income (k$)'].mean(),data['Spending Score (1-100)'].mean())
```

```
In [ ]:
```

```
In [ ]:
```